

Who Owns Models and Data in the Cluster

Which node holds and needs the models, the documents and the results in every execution mode; whether a node without the Spark master is supported; and what to change for a multi-machine air-gapped installation.

item	value
Questions answered	1. In a clustered environment, which node owns the models and the data? 2. One node does not have the master - does the repository support that as it is, and what needs to change in the air-gapped system?
Based on	The code paths that open models and files: inference/text_pipeline_engine.py (run_text_pipeline_job, run_text_pipeline_job_via_service), inference/cluster_engine.py (run_cluster_inference), submit_pipeline_job.py, submit_job.py, models/model_store.py, models/pipelines/manifest.json; the traced runs and the 12 air-gap simulation tests
Audience	Engineers planning where models, documents and services go on the air-gapped machines
Related	NER_SPARK_DATA_PATHS_AND_EXECUTION.pdf (every data path, measured), CLUSTER_RUNBOOK_MODES_AND_TESTS.pdf, WINDOWS_AND_UBUNTU_SUPPORT.docx, AIRGAP_DEPENDENCIES_AND_VERSIONS.pdf

1. The short answer	2
2. What each node holds and needs	2
3. Where each kind of data lives	3
4. Check (a): a node that does not run the Spark master	3
5. Check (b): a node that cannot see the master's file system	4
5.1 Models without the master's folder	4
5.2 Documents without a share - the optional code change	4
6. Changes for a multi-machine air-gapped installation.....	4
T1 - Make the manifest's names resolvable	4
T2 - HDFS datanode address	5
T3 - One shared data folder, same path everywhere it is used	5
T4 - Model store settings on model-loading nodes	5
T5 - Results and ports	5
7. Example: four machines, service mode	5
8. Checklist.....	6

1. The short answer

- **The Spark master owns nothing.** It only keeps the list of workers and hands out executors. It never opens a model or a document.
- **Models are owned by the model store** - HDFS, or a shared folder on any machine - and are loaded only by the process that runs them: the model server (service mode) or every Spark worker (cluster mode). Each of those nodes keeps its own local cache after the first load. For tensor models (plugins) the driver loads the weights and ships them to the executors.
- **Documents are owned by a shared data folder** (an NFS export or similar, on any machine). Only two kinds of process touch it: the driver, which lists the files, and whoever runs the models, which opens them.
- **Results are written by the driver** into its results/ folder.

question	answer	change needed?
(a) One node does not run the Spark master	Supported as it is: a cluster has exactly one master; every other node runs only workers (or the model server, or storage). The master can also share a machine with a worker.	no
(b) One node cannot see the master's file system	Models: supported through HDFS (nodes fetch over the network). Documents: supported for Spark workers in service mode, which never open files. The driver and the model server must see the documents, and so must every worker in cluster mode.	only if a document-reading node cannot mount the share (section 5)
Several machines in the air gap	The names in models/pipelines/manifest.json (ner-translate-master, ner-translate-server) and the HDFS datanode name come from Docker's own network and do not exist across machines.	yes - name resolution, no code change (section 6)

2. What each node holds and needs

By execution mode. "Share" = the shared data folder mounted at the same path (/app/data inside the containers); "model store" = network access to HDFS (WebHDFS) or the shared model folder.

node role	NER - service mode (recommended)	NER - cluster mode	tensor models (plugins)
Spark master	nothing	nothing	nothing
Driver (runs where you submit - by convention in the master container)	the share (lists the files) writes results/ no models (spark-lean image, no torch)	the share (lists the files) writes results/ the NER image: imports the pipeline code, loads no weights	the model weights (loads them; plugin weights_path may be hdfs://) the input data (.npy, or generated) writes results/
Spark worker / executors	nothing but network: sends file names to the model server; no share, no models, no GPU	the share (opens the documents) model store + local model cache (>= 5 GB) GPU; loads GLiNER + NLLB per task	nothing on disk: the driver broadcasts the weights and parallelises the data; GPU for gpu / hybrid modes
Model server (kitchen)	the share (opens the documents) model store + local model cache GPU; the only loaded copy of the models	not used	not used (Triton plays this role in the Triton mode)
Storage node	HDFS: model files NFS: documents	same	optional: plugin weights in HDFS
Backend (Django gateway)	writes uploads into the share	same	.npy uploads into the share (the driver reads them)

31. Who owns models and data - recommended air-gapped layout (service mode)

Each box is a machine. The Spark master owns nothing: models live in the model store, documents in the shared folder

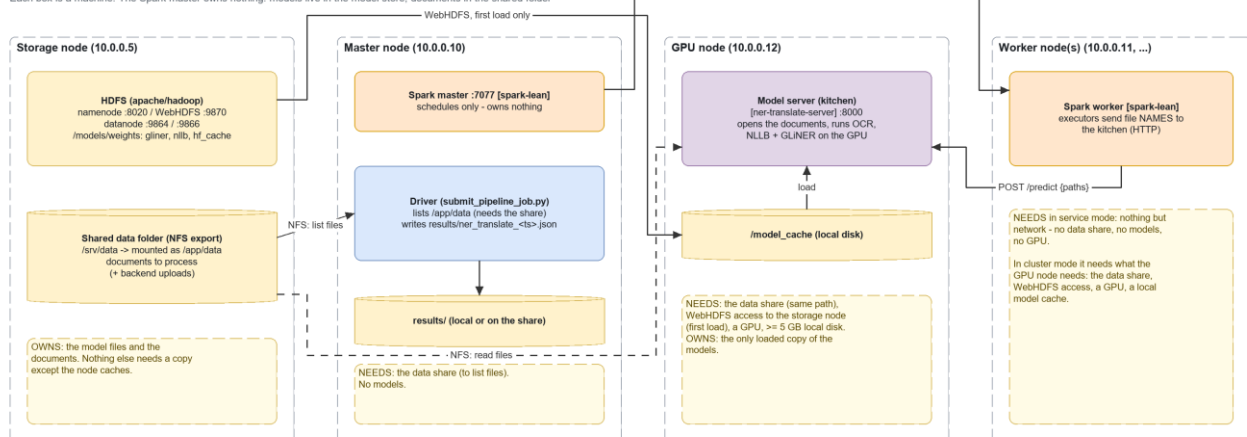


Figure 1 - recommended air-gapped layout for service mode: who owns what, who reads what

Why it works this way (from the code): in service mode the executors run `run_text_pipeline_job_via_service`, whose task only POSTs the list of path strings to the model server - no file is opened on a worker. In cluster mode the task calls the pipeline's `load()` and `run()`, which fetch the models into the node cache and open every document. For tensor models `run_cluster_inference` serialises each model's `state_dict` on the driver and broadcasts it, and `parallelize` sends the input rows with the tasks.

3. Where each kind of data lives

data	lives on	written by	read by	lifetime
model files (gliner-multi, nlb-200-distilled-600M, hf_cache)	HDFS /models/weights (or a shared folder)	you, once (hdfs dfs - put)	model server / cluster-mode workers, first load only	until you replace them; a new version goes into a new folder
model caches	local disk of each model-loading node (/model_cache)	models/model_store.py	the same node	survive restarts; delete to force a fresh fetch
documents	the shared data folder	users / the Django backend (uploads/)	driver (listing), model server or cluster-mode workers	your retention policy
results	results/ on the driver node (or on the share)	the driver	you / the backend	your retention policy
logs	Spark work dirs on each worker; container logs on each node	Spark, Docker	operators	until cleaned
code	baked into the images; mounted from the platform folder	releases	every container	per release

4. Check (a): a node that does not run the Spark master

Supported without changes. A standalone Spark cluster has exactly one master; the other nodes run workers (and, separately, the model server or storage). Nothing in the repository assumes the master is on a particular machine:

- Workers join with `start-worker.sh spark://<master-ip>:7077`; the submit scripts take `--master spark://<master-ip>:7077` (or `SPARK_MASTER_URL`), which overrides the manifest's `master_url`.
- The driver usually runs inside the master container (`docker exec <master> python submit_...py`), but it can run on any machine with the same image and network reach: executors connect back to the driver, not to the master.
- The master holds no data, so a small CPU-only machine is enough. It may also share a machine with a worker or the model server (that is what the single-machine setups do).

What every node must be able to reach: the master on 7077, the driver's RPC and BlockManager ports (random unless fixed with `spark.driver.port` / `spark.blockManager.port`), and - in service mode - the model server on 8000.

5. Check (b): a node that cannot see the master's file system

"The master's file system" does not have to be on the master at all: models can come from HDFS and documents from a share exported by any machine (the storage node in Figure 1). What each role needs:

role of the node without the share	needs documents?	needs models?	supported now?	if not, the change
Spark worker, service mode	no	no	yes	-
Spark worker, tensor models	no (data comes with the tasks)	no (weights are broadcast)	yes	-
Model server (kitchen)	yes - opens them	yes - via HDFS works without any share	models: yes (HDFS) documents: needs the share	mount the share on this node
Spark worker, cluster mode	yes - opens them	yes - via HDFS	models: yes (HDFS) documents: no	mount the share; or run this node as a service-mode worker; or the code change below
Driver	yes - lists them	no (NER) / yes (tensor)	needs the share	run the driver on a node that has it

5.1 Models without the master's folder

Two model-store modes exist; each container picks its own through its environment, so they can be mixed per node:

- `MODEL_STORE_URI=hdfs://<namenode>:8020/models/weights` - the node fetches over WebHDFS into its local cache. No shared folder needed. **Use this for any node that cannot mount the master's folder.**
- `MODEL_STORE_URI=file:///mnt/models` - the node reads a folder mounted read-only at `/mnt/models`. Every node that loads models needs that mount.

5.2 Documents without a share - the optional code change

If a document-reading node (model server or cluster-mode worker) can never mount the share, documents would have to come from HDFS too. That is **not implemented today** - input paths are local file paths. The change would be small and reuse the model store's WebHDFS client:

- `submit_pipeline_job.py`: accept `--input hdfs://.../incoming`, list the folder with WebHDFS (LISTSTATUS) instead of glob, and keep `hdfs://` paths as they are (no abspath).
- `mt_ner_all_formats.extract_text`: when a path starts with `hdfs://`, download it to a temporary file first (`models/model_store.py` already has the download code), extract, delete the temporary file.
- Results stay keyed by file name; the model server would need WebHDFS access to the documents' datanodes as well.

6. Changes for a multi-machine air-gapped installation

The simulation runs every service on one Docker network, where names like `ner-translate-server` and `hdfs-datanode` resolve automatically. On separate machines they do not. None of the following needs a code change or an image rebuild.

T1 - Make the manifest's names resolvable

`models/pipelines/manifest.json` points at `spark://ner-translate-master:7077` and `http://ner-translate-server:8000`. With `--execution-mode auto` the driver uses service mode only if that name resolves where it runs, and in every service-mode run **each executor** POSTs to it. Either map the names on every container that runs the driver or an executor:

```
docker run ... --add-host ner-translate-server:10.0.0.12 --add-host ner-translate-master:10.0.0.10 ...
# compose: extra_hosts: ["ner-translate-server:10.0.0.12", "ner-translate-master:10.0.0.10"]
# or the same two lines in /etc/hosts of every machine (host-network containers use the host's names)
```

or edit `models/pipelines/manifest.json` (it is in the mounted `models/` folder, so no rebuild) to use real host names or IPs. Always pass `--execution-mode service` explicitly: with `auto`, an unresolvable name silently falls back to in-process execution, which fails on the lean image.

T2 - HDFS datanode address

A WebHDFS read goes to the namenode, which redirects it to a datanode by the datanode's configured host name (`dfs.datanode.hostname`, set to the Docker name `hdfs-datanode` in the simulation). Set it to a name or IP every model-loading node can resolve, and open the namenode (8020, 9870) and datanode (9864, 9866) ports to those nodes.

T3 - One shared data folder, same path everywhere it is used

```
# storage node 10.0.0.5: export the folder
echo "/srv/data 10.0.0.0/24(rw, sync, no_subtree_check)" | sudo tee -a /etc/exports && sudo exportfs -ra
# driver node and model-server node (and cluster-mode workers): mount it at the same path
sudo mount -t nfs 10.0.0.5:/srv/data /srv/data          # then: docker run ... -v /srv/data:/app/data ...
```

The Django backend's `SHARED_DATA_DIR` must be the same share, so uploads are visible to the driver and the model server.

T4 - Model store settings on model-loading nodes

`MODEL_STORE_URI=hdfs://10.0.0.5:8020/models/weights`, `MODEL_CACHE_DIR=/model_cache` on a named volume on local disk (>= 5 GB free). Only the model server (service mode) or the workers (cluster mode) need these.

T5 - Results and ports

- Results land in `results/` on the driver node; mount `results/` from the share if they should be collected centrally.
- Open between the machines: 7077 / 8080 (master), 8081 + worker port (workers), driver RPC + BlockManager ports, 8000 (model server), 8020 / 9870 / 9864 / 9866 (HDFS).
- Run the same image on the driver and the executors of a mode (spark-lean for service mode, ner-translate-worker for cluster mode), so Python and package versions match.

7. Example: four machines, service mode

machine	runs	mounts	needs to reach
storage 10.0.0.5	HDFS namenode + datanode (<code>apache/hadoop</code> , host network, <code>dfs.datanode.hostname=10.0.0.5</code>); NFS export <code>/srv/data</code>	-	-
master 10.0.0.10	Spark master + driver [spark-lean]	<code>/srv/data -> /app/data</code> ; <code>results/</code>	10.0.0.12:8000
GPU 10.0.0.12	model server [ner-translate-server]	<code>/srv/data -> /app/data</code> ; platform models/ (code) -> <code>/app/models</code> ; model-cache volume	HDFS 10.0.0.5:9870 / 9864
worker 10.0.0.11 (and more)	Spark worker [spark-lean]	nothing	10.0.0.10:7077, the driver, 10.0.0.12:8000

```
# master 10.0.0.10
docker run -d --name ner-translate-master --network host -e SPARK_LOCAL_IP=10.0.0.10 \
  --add-host ner-translate-server:10.0.0.12 -v /srv/data:/app/data -v /opt/platform/results:/app/results \
  spark-lean:latest bash -c "start-master.sh -h 10.0.0.10 && tail -f /opt/spark/logs/*master*"

# GPU 10.0.0.12 - the model server (its code comes from the mounted models/ folder)
docker run -d --name ner-translate-server --network host --gpus all --init \
  -e MODEL_STORE_URI=hdfs://10.0.0.5:8020/models/weights -e MODEL_CACHE_DIR=/model_cache \
  -v /srv/data:/app/data -v /opt/platform/models:/app/models -v model-cache:/model_cache \
  ner-translate-server:latest

# worker 10.0.0.11 - no mounts at all in service mode
docker run -d --name ner-translate-worker --network host -e SPARK_LOCAL_IP=10.0.0.11 \
  --add-host ner-translate-server:10.0.0.12 \
  spark-lean:latest bash -c "start-worker.sh spark://10.0.0.10:7077 -c 4 -m 4g && tail -f \
  /opt/spark/logs/*worker*"

# submit (on the master)
docker exec ner-translate-master python submit_pipeline_job.py --pipeline ner_translate \
  --input /app/data/incoming --execution-mode service --master spark://10.0.0.10:7077 --partitions 4
```

For cluster mode instead, the worker machines run the `ner-translate-worker` image with a GPU, the `/srv/data` mount, the `MODEL_STORE_URI` / `MODEL_CACHE_DIR` settings and a model-cache volume, and the model server is not needed.

8. Checklist

check	how	expected
manifest names resolve on the driver and every executor	<code>docker exec <container> getent hosts ner-translate-server</code>	the model server's IP
model server reachable from a worker	<code>docker exec <worker> python -c "import urllib.request;print(urllib.request.urlopen('http://ner-translate-server:8000/health').read())"</code>	<code>{"status":"ok"}</code>
share visible at the same path	<code>ls /app/data/incoming</code> in the driver and the model server containers	the same files
HDFS reachable from the model-loading nodes	<code>curl -s http://10.0.0.5:9870/webhdfs/v1/models/weights?op=LISTSTATUS</code>	gliner-multi, nllb-200-distilled-600M, hf_cache
first model load	model server log	[model_store] fetched ... then cache hit on restarts
end to end	the submit command of section 7	N document(s) processed, results/ner_translate_<ts>.json on the master